AquaSense AI

AI-Based IoT Water Management System

Smart Water Monitoring • AI Leak Detection • Intelligent Pump Control

Java Spring Boot • ESP32 • Machine Learning • MQTT • MySQL • MIT License

Md. Ashikur Rahman Ashik
Department of Information and Communication Technology (ICT)
Mawlana Bhashani Science and Technology University (MBSTU)

https://github.com/Ashikur758/AquaSense-AI

Contents

1 Overview **2**

2 Project Objectives **2**

3 Key Features **2**

4 Project Screenshots **2**

5 System Architecture **2**

6 Technology Stack **5**

7 Hardware Components **5**

8 Circuit Connections **5**

9 Project Structure **5**

10 AI Modules **6**
10.1 AI-Based Leakage Detection 6
10.2 Water Demand Forecasting 7
10.3 Intelligent Pump Control 7

11 REST API Endpoints **7**

12 MQTT Communication **8**

13 Getting Started **8**
13.1 Prerequisites 8
13.2 Clone the Repository 8
13.3 Create the Database 9
13.4 Configure Backend 9
13.5 Run Spring Boot Backend 9
13.6 Configure ESP32 9
13.7 Open Web Dashboard 9

14 Future Improvements **9**

15 Contributing **10**

16 License **10**

17 Author **10**

1 Overview

AquaSense AI is an AI-powered IoT-based water management system designed to monitor water consumption, detect leakage, predict future water demand, and control water pumps intelligently.

The system combines IoT sensors, ESP32, Java Spring Boot, MQTT communication, and Machine Learning to provide an intelligent water monitoring and management solution.

It collects real-time water data from sensors and sends it to a centralized backend server. The backend processes the data, stores historical records, and uses AI-based algorithms to identify abnormal water usage patterns.

Users can monitor their water system and control pumps through a web-based dashboard.

2 Project Objectives

- Reduce water wastage.
- Monitor water levels in real time.
- Detect abnormal water flow and leakage.
- Predict future water consumption.
- Automate water pump operations.
- Provide remote monitoring and control.
- Improve water resource management.

3 Key Features

Feature	Description
Real-Time Monitoring	Monitor water level, flow rate, and consumption.
Smart Pump Control	Automatically control pumps based on water levels.
AI Leakage Detection	Identify abnormal water flow patterns.
Demand Forecasting	Predict future water requirements.
Intelligent Scheduling	Optimize pump operation schedules.
Alert System	Notify users about leakage, overflow, and low water levels.
Secure Authentication	JWT-based login and role management.
Web Dashboard	Visualize sensor data and manage devices remotely.

4 Project Screenshots

Note: Place your actual project images in `docs/images/` using the filenames shown above. Placeholders are displayed until the images exist.

5 System Architecture

The system consists of four major layers:

1. **IoT Layer:** ESP32 collects sensor data.

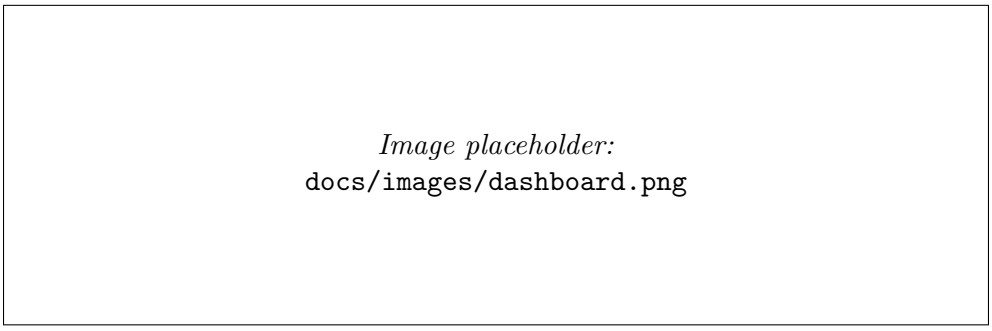


Figure 1: Web Dashboard: real-time water monitoring and intelligent control dashboard.

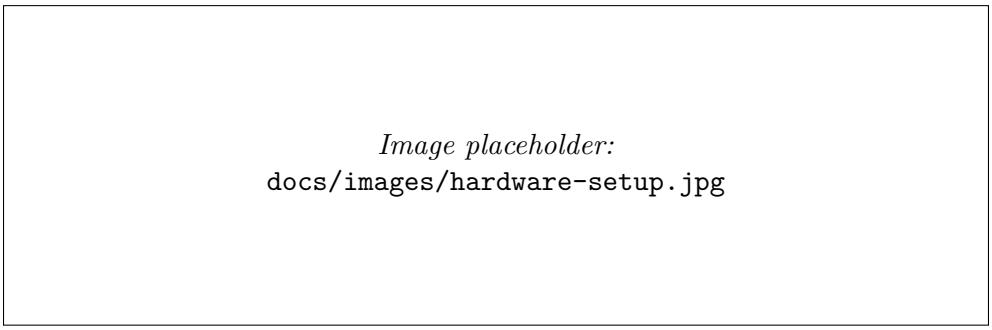


Figure 2: Hardware Implementation: ESP32-based IoT water monitoring hardware.

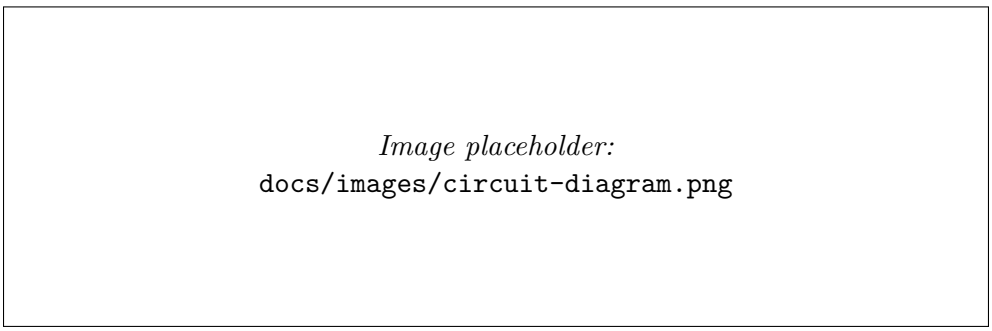


Figure 3: Circuit Diagram.

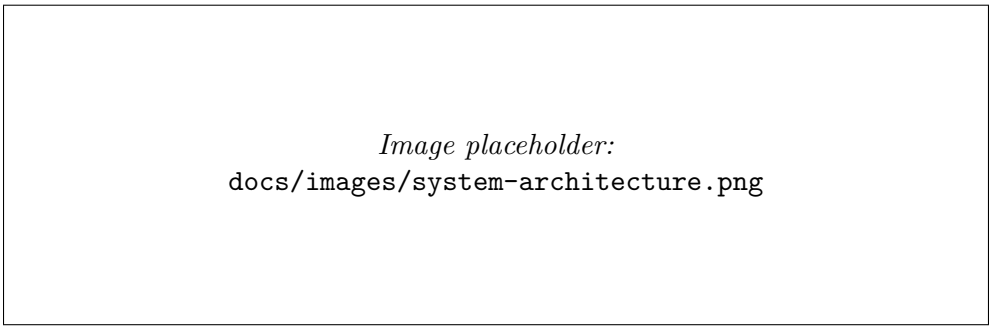


Figure 4: System Architecture.

Image placeholder:
`docs/images/leakage-alert.png`

Figure 5: Leakage Detection Alert.

2. **Communication Layer:** MQTT transfers data between ESP32 and the server.
3. **Backend & AI Layer:** Spring Boot processes data and executes AI algorithms.
4. **Presentation Layer:** Web dashboard displays information and provides control.

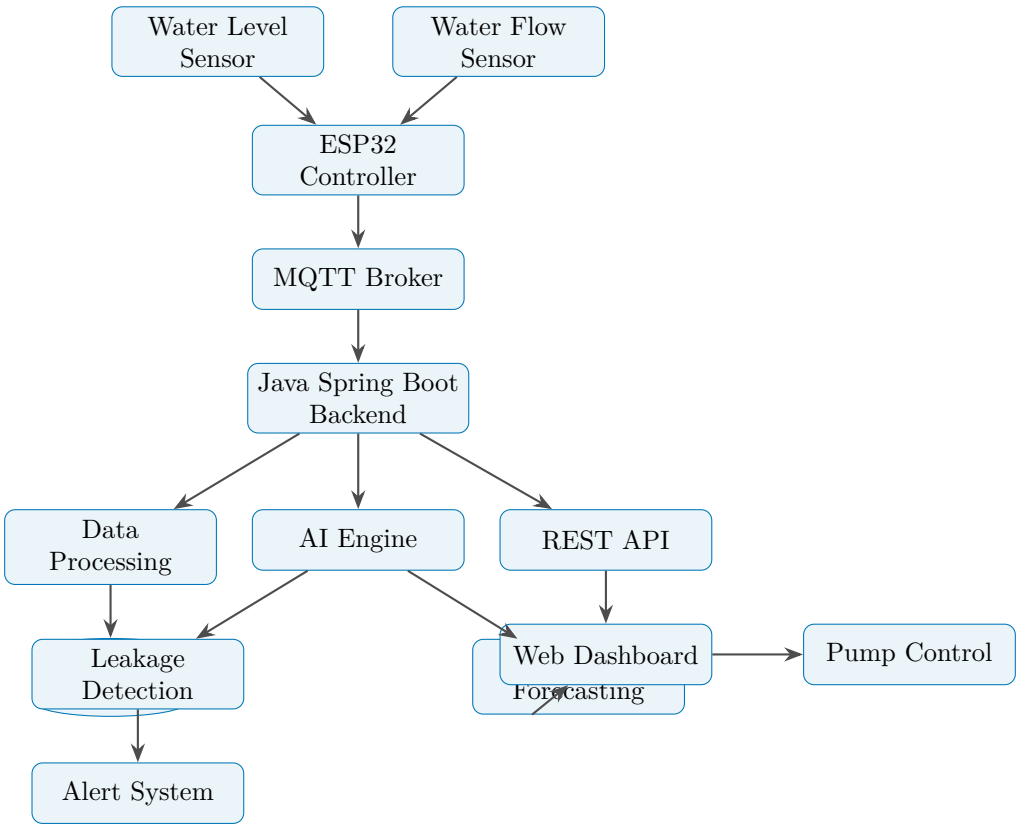


Figure 6: Data flow of the AquaSense AI system. Pump commands travel from the dashboard back through the MQTT broker to the ESP32, which drives the relay module and water pump.

6 Technology Stack

Layer	Technologies
Embedded / IoT	ESP32 DevKit, Arduino IDE / PlatformIO, Embedded C/C++, Ultrasonic Sensor, Water Flow Sensor, MQTT Communication
Backend	Java 17+, Spring Boot, Spring Web, Spring Data JPA, Spring Security, Spring WebSocket, Eclipse Paho MQTT Client
AI & Machine Learning	Statistical Anomaly Detection, Z-Score Algorithm, Moving Average, Linear Regression, Polynomial Regression, Weka / Smile / Tribuo
Frontend	HTML5, CSS3, JavaScript, Bootstrap, Chart.js, Thymeleaf / React
Database & Communication	MySQL / PostgreSQL, MQTT, Mosquitto Broker, HTTP REST API

7 Hardware Components

Component	Purpose
ESP32 DevKit	Main microcontroller
HC-SR04 Ultrasonic Sensor	Measures tank water level
YF-S201 Flow Sensor	Measures water flow
Relay Module	Controls water pump
Water Pump	Pumps water into the tank
Power Supply	Provides electrical power
Connecting Wires	Hardware connections
TDS Sensor (Optional)	Measures water quality
pH Sensor (Optional)	Monitors water acidity

8 Circuit Connections

The following pin configuration is suggested for the ESP32.

Device	Pin	ESP32 GPIO
HC-SR04	TRIG	GPIO 5
HC-SR04	ECHO	GPIO 18
YF-S201	Signal	GPIO 27
Relay Module	IN	GPIO 26
All Sensors	GND	GND
Sensors	VCC	Appropriate supply

Important: The HC-SR04 ECHO pin produces a 5 V signal. Use a voltage divider before connecting it to the ESP32 GPIO. Check the voltage requirements of all modules before powering them.

9 Project Structure

```
AquaSense-AI/
|
|-- firmware/
|   |-- src/
|   |   `-- main.ino
|   `-- README.md
|
|-- backend/
|   `-- src/main/java/com/aquasense/
|       |-- controller/
|       |-- service/
|       |-- repository/
|       |-- model/
|       |-- mqtt/
|       |-- ai/
|       |-- security/
|       `-- AquaSenseApplication.java
|
|-- frontend/
|   |-- css/
|   |-- js/
|   `-- index.html
|
|-- docs/
|   `-- images/
|       |-- banner.png
|       |-- dashboard.png
|       |-- hardware-setup.jpg
|       |-- circuit-diagram.png
|       |-- system-architecture.png
|       `-- leakage-alert.png
|
|-- .gitignore
|-- LICENSE
`-- README.md
```

10 AI Modules

10.1 AI-Based Leakage Detection

The leakage detection module analyzes water flow patterns to identify abnormal consumption.

Working process:

- Collect real-time flow sensor readings.
- Calculate rolling mean and standard deviation.
- Apply Z-Score anomaly detection.
- Identify unusual water consumption.
- Generate leakage alerts.

The anomaly score is computed as

$$z = \frac{x_t - \mu}{\sigma},$$

where x_t is the current flow, μ the rolling mean, and σ the rolling standard deviation. A reading is flagged when $|z| > 3$.

```
double zScore = (currentFlow - mean) / standardDeviation;

if (Math.abs(zScore) > 3.0) {
    System.out.println("Abnormal water flow detected!");
}
```

10.2 Water Demand Forecasting

The forecasting module predicts future water consumption using historical data.

Working process:

- Collect historical water consumption.
- Analyze daily and hourly usage.
- Apply regression algorithms.
- Predict upcoming water demand.
- Optimize water storage and pump scheduling.

10.3 Intelligent Pump Control

The smart pump control module considers:

- Current tank water level.
- Predicted water demand.
- Historical consumption.
- Pump operating status.

It automatically controls the pump to reduce unnecessary energy consumption and water wastage.

Note: AI predictions are decision-support inputs. Independent safety limits and sensor checks should always protect against overflow and dry running.

11 REST API Endpoints

Method	Endpoint	Description
POST	`/api/auth/register`	Register a new user
POST	`/api/auth/login`	User authentication
POST	`/api/devices`	Register an IoT device
GET	`/api/devices`	Get all devices
POST	`/api/sensor-data`	Receive sensor readings
GET	`/api/devices/{id}/latest`	Get latest sensor data
GET	`/api/devices/{id}/history`	Get historical data
POST	`/api/devices/{id}/pump`	Control water pump
GET	`/api/devices/{id}/forecast`	Get AI demand forecast
GET	`/api/alerts`	Get system alerts

Sample Sensor Data (JSON)

```
{
  "deviceId": "ESP32-001",
  "waterLevelPercent": 72.5,
  "flowRateLpm": 3.4,
  "totalLiters": 1250.8,
  "pumpStatus": false,
  "timestamp": "2026-01-15T10:30:00Z"
}
```

12 MQTT Communication

MQTT is used for communication between the ESP32 and the Spring Boot backend.

MQTT Topic	Direction	Purpose
`aquasense/{deviceId}/data`	ESP32 → Server	Sensor readings
`aquasense/{deviceId}/command`	Server → ESP32	Pump control
`aquasense/{deviceId}/status`	ESP32 → Server	Device status

Example Pump Command

```
{
  "command": "PUMP_ON"
}
```

13 Getting Started

13.1 Prerequisites

Install the following software:

- Java JDK 17+
- IntelliJ IDEA
- Apache Maven
- MySQL
- Arduino IDE
- Mosquitto MQTT Broker
- Git

13.2 Clone the Repository

```
git clone https://github.com/Ashikur758/AquaSense-AI.git

cd AquaSense-AI
```

13.3 Create the Database

```
CREATE DATABASE aquasense;
```

13.4 Configure Backend

Open `backend/src/main/resources/application.properties` and add:

```
spring.datasource.url=jdbc:mysql://localhost:3306/aquasense
spring.datasource.username=your_username
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update

mqtt.broker.url=tcp://localhost:1883

server.port=8080
```

Configure your JWT secret securely using environment variables rather than committing secrets to GitHub.

13.5 Run Spring Boot Backend

```
cd backend

./mvnw spring-boot:run
```

On Windows:

```
mvnw.cmd spring-boot:run
```

Backend URL: `http://localhost:8080`

13.6 Configure ESP32

1. Open `firmware/src/main.ino`.
2. Configure Wi-Fi credentials.
3. Configure MQTT broker address.
4. Connect the sensors according to the pin configuration.
5. Select *ESP32 Dev Module* in Arduino IDE.
6. Upload the firmware.

13.7 Open Web Dashboard

Visit `http://localhost:8080` and log in to access the AquaSense AI dashboard.

14 Future Improvements

- ☐ Complete project architecture.
- ☐ ESP32 water-level monitoring.
- ☐ Water flow monitoring.

- ☐ ESP32 to Spring Boot communication.
- ☐ MQTT integration.
- ☐ MySQL database integration.
- ☐ Automatic pump control.
- ☐ Real-time web dashboard.
- ☐ AI-based leakage detection.
- ☐ Water demand forecasting.
- ☐ Smart pump scheduling.
- ☐ Email and SMS notifications.
- ☐ Water quality monitoring.
- ☐ Mobile application.
- ☐ Cloud deployment using Docker.

15 Contributing

Contributions are welcome! To contribute:

1. Fork this repository.
2. Create a new branch:
   ```
   git checkout -b feature/your-feature
   ```
3. Commit your changes:
   ```
   git commit -m "Add new feature"
   ```
4. Push your branch:
   ```
   git push origin feature/your-feature
   ```
5. Open a Pull Request.

16 License

This project is licensed under the MIT License. See the `LICENSE` file for more information.

17 Author

Md. Ashikur Rahman Ashik
Department of Information and Communication Technology (ICT)
Mawlana Bhashani Science and Technology University (MBSTU)

- GitHub: @Ashikur758
- Project Repository: AquaSense AI

AquaSense AI – Smart Technology for a Sustainable Water Future.